



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A' Grade

Department of Computer Science and Engineering DevOps Lab - L3

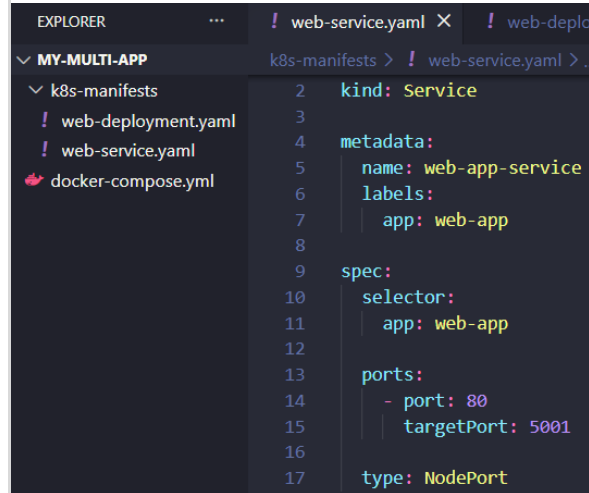
Lab Assignment No: 13	
Name of Student	Sanyukt Ramola
PRN	23070122194
Batch	2023 - 2027
Class	CSE – B1
Academic Year & Semester	2026 - 2027, 7th Semester
Date of Performance	30 th July, 2026
Title of Assignment	Kubernetes Service & Networking
Theory	A Kubernetes Service is an abstraction that provides a stable network endpoint for accessing one or more Pods in a Kubernetes cluster. Since Pods are temporary and their IP addresses may change, a Service uses labels and selectors to route incoming traffic to the appropriate Pods. A NodePort Service exposes the application outside the cluster by allocating a port on each cluster node, allowing external access to the application. During local development, <code>kubectl port-forward</code> can also be used to securely forward traffic from a local machine to a Kubernetes Service for testing and verification. This approach simplifies communication between applications and ensures reliable access to containerized workloads. Commands Used • Created the Kubernetes Deployment: <code>kubectl apply -f web-deployment.yaml</code> • Verified that the Pods were running: <code>kubectl get pods</code> • Created the Kubernetes Service: <code>kubectl apply -f web-service.yaml</code> • Displayed the available Kubernetes Services: <code>kubectl get service</code> • Displayed detailed information about the Service: <code>kubectl describe service web-app-service</code> • Displayed the Service endpoints: <code>kubectl get endpoints</code>

- Displayed the running Pods with labels:
kubectl get pods --show-labels
- Displayed the application logs:
kubectl logs web-app-deployment-<pod-name>
- Forwarded the Kubernetes Service to the local machine:
kubectl port-forward service/web-app-service 8085:80
- Deleted the Kubernetes Service:
kubectl delete -f web-service.yaml

Deleted the Kubernetes Deployment:
kubectl delete -f web-deployment.yaml

Implementation

Step 1: Create the Kubernetes Service YAML



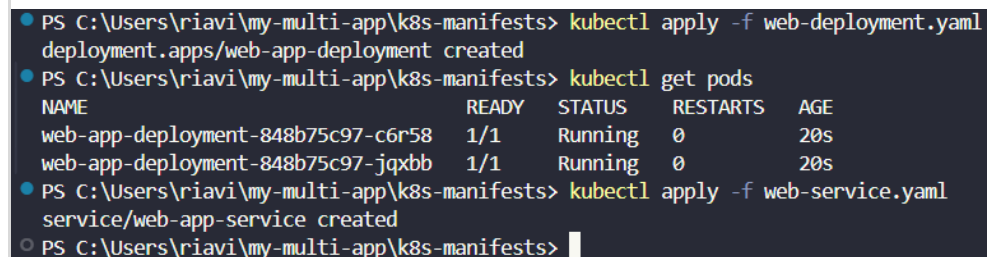
```

EXPLORER
...
! web-service.yaml X
! web-deplo

MY-MULTI-APP
k8s-manifests > ! web-service.yaml > ...
  k8s-manifests
    ! web-deployment.yaml
    ! web-service.yaml
    docker-compose.yml
  2 kind: Service
  3
  4 metadata:
  5   name: web-app-service
  6   labels:
  7     app: web-app
  8
  9 spec:
 10   selector:
 11     app: web-app
 12
 13   ports:
 14     - port: 80
 15       targetPort: 5001
 16
 17   type: NodePort
  
```

Created Kubernetes Service Manifest — Created the web-service.yaml file defining a Kubernetes NodePort Service to expose the Flask web application running in the Kubernetes Pods.

Step 2: Apply the Service

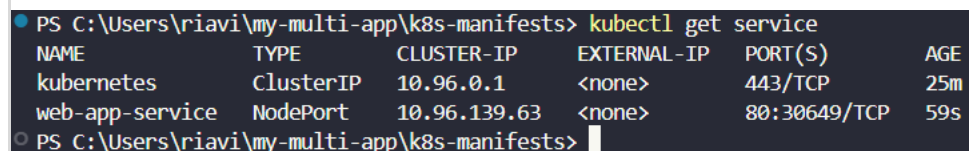


```

PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl apply -f web-deployment.yaml
deployment.apps/web-app-deployment created
PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
web-app-deployment-848b75c97-c6r58  1/1     Running   0           20s
web-app-deployment-848b75c97-jqxbb  1/1     Running   0           20s
PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl apply -f web-service.yaml
service/web-app-service created
PS C:\Users\riavi\my-multi-app\k8s-manifests>
  
```

Applied Kubernetes Service — Applied the Kubernetes Service manifest using kubectl apply, creating a NodePort Service for the deployed Flask application.

Step 3: Find the Service Information



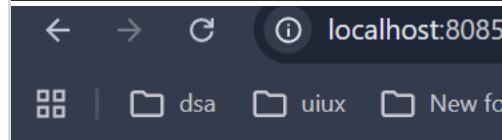
```

PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl get service
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
kubernetes      ClusterIP   10.96.0.1     <none>         443/TCP          25m
web-app-service NodePort    10.96.139.63  <none>         80:30649/TCP    59s
PS C:\Users\riavi\my-multi-app\k8s-manifests>
  
```

Verified Kubernetes Service — Verified the Kubernetes NodePort Service and identified the assigned NodePort (30649) used to expose the Flask web application outside the Kubernetes cluster.

Step 4: Access the Application

```
PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl port-forward service/web-app-service 8085:8085
Forwarding from 127.0.0.1:8085 -> 5001
Forwarding from [::1]:8085 -> 5001
Handling connection for 8085
Handling connection for 8085
```



Hello, Docker!

Log:

2026-07-22 14:59:23: Hello, Docker!

Accessed Application Through Kubernetes Service — Used kubectl port-forward to forward the Kubernetes Service to the local machine and successfully accessed the Flask web application at <http://localhost:8085>, verifying that it was running correctly.

Step 5: Clean Up

```
PS C:\Users\riavi\my-multi-app\k8s-manifests> kubectl delete -f web-service.yaml
service "web-app-service" deleted from default namespace
PS C:\Users\riavi\my-multi-app\k8s-manifests>
```

Deleted Kubernetes Service — Removed the Kubernetes Service using the kubectl delete command, cleaning up the networking resources created for the application.

Subject Faculty Name: Prof. Arijit Dutta